CHATBOT CIMA

FASE II / VALIDACIÓN Y PRUEBAS

Resultados de integración continua, construcción de contenedor, despliegue y pruebas funcionales de API e interfaz.

Aplicación	CHATBOT CIMA — clasificador NLP de consultas operativas
Despliegue	Render · Docker · FastAPI
Fecha de corte	31 de octubre de 2026

URL pública: https://chatbot-cima-fase2.onrender.com/

https://github.com/alfredosalazar-ai/deep-learning-experiments

1. Resumen ejecutivo

La Fase II de CHATBOT CIMA implementa y despliega en la nube un prototipo académico de clasificación de consultas operativas mediante procesamiento de lenguaje natural. La aplicación integra una interfaz web, una API desarrollada con FastAPI y el Pipeline de Regresión Logística entrenado en la Actividad 5. El enrutamiento aplica un umbral operativo de 0.276334 para la puntuación de la clase INDIVIDUAL y deriva a revisión humana los casos que no cumplen la política definida.

Resultado de infraestructura GitHub Actions completó las pruebas, construyó la imagen Docker y probó el contenedor. Render confirmó “Deploy succeeded / Live”; los logs registran la carga del modelo y respuestas HTTP 200 de /health.

La validación funcional incluyó cuatro consultas desde la interfaz web y ocho pruebas desde Swagger UI. Las pruebas de entrada inválida respondieron con HTTP 422 según lo esperado. En las pruebas de clasificación se observaron dos discrepancias entre la intención de la consulta y la clase predicha; se conservaron como hallazgos y no se reinterpretaron como aciertos.

2. Alcance y criterios

- Validación de disponibilidad de la API y carga del artefacto del modelo.
- Pruebas de contrato API: cuerpo de solicitud, respuesta JSON, códigos HTTP y restricciones de longitud.
- Pruebas de clasificación con consultas sintéticas para las cuatro categorías del proyecto.
- Validación de la canalización CI/CD, construcción Docker y ejecución del contenedor en infraestructura remota.
- Comprobación de que la predicción no se presente como autorización de pago ni sustituya controles deterministas o revisión humana.

Fuera de alcance Este documento no es una evaluación estadística nueva del modelo ni acredita desempeño en datos productivos. Los resultados funcionales son pruebas sintéticas de una versión desplegada.

3. Arquitectura y entorno probado

Componente	Implementación / configuración
Repositorio	GitHub: alfredosalazar-ai/deep-learning-experiments · rama main
Backend	FastAPI 0.115.12 · Uvicorn 0.34.0
Modelo	Regresión Logística · Pipeline de scikit-learn 1.6.1
Versión reportada por la API	actividad5-regresion-logistica-a5-v1.0
Runtime del contenedor	Python 3.12-slim
Dependencias numéricas	NumPy 1.26.4 · SciPy 1.15.2 · joblib 1.6.0
Frontend	HTML, CSS y JavaScript servidos por FastAPI

Componente	Implementación / configuración
Contenedor	Docker construido y ejecutado por GitHub Actions y Render
Despliegue	Render Web Service · Free · Oregon (US West)
URL pública	https://chatbot-cima-fase2.onrender.com/
Swagger UI	https://chatbot-cima-fase2.onrender.com/docs
Health Check	https://chatbot-cima-fase2.onrender.com/health

4. Integración continua y construcción Docker

Se observaron tres ejecuciones exitosas del workflow “Fase II - prueba y construcción Docker”. La tercera ejecución corresponde al commit “Update GitHub Actions runtimes and pin Ubuntu runner”, con runner ubuntu-24.04 y acciones actions/checkout@v7 y actions/setup-python@v7. Esta actualización eliminó las advertencias de compatibilidad de Node.js 20 y evitó depender de la etiqueta móvil ubuntu-latest.

Ejecución	Resultado	Duración visible	Observación
Docker FASE II #1	Success	52 s (run summary)	Pruebas, build Docker y prueba del contenedor completados.
Improve health check readiness validation #2	Success	54 s	Se mantuvo verde tras ajustar /health para reflejar disponibilidad del modelo.
Update GitHub Actions runtimes and pin Ubuntu runner #3	Success	1 min 3 s	ubuntu-24.04; checkout@v7; setup-python@v7; todos los pasos en verde.

En la ejecución #3 se confirmaron los pasos de instalación de dependencias, pytest de API y modelo, construcción de la imagen Docker y prueba del contenedor construido. Esta validación demuestra que el artefacto se puede construir y ejecutar en un runner remoto; no requiere Docker instalado en el equipo del usuario.

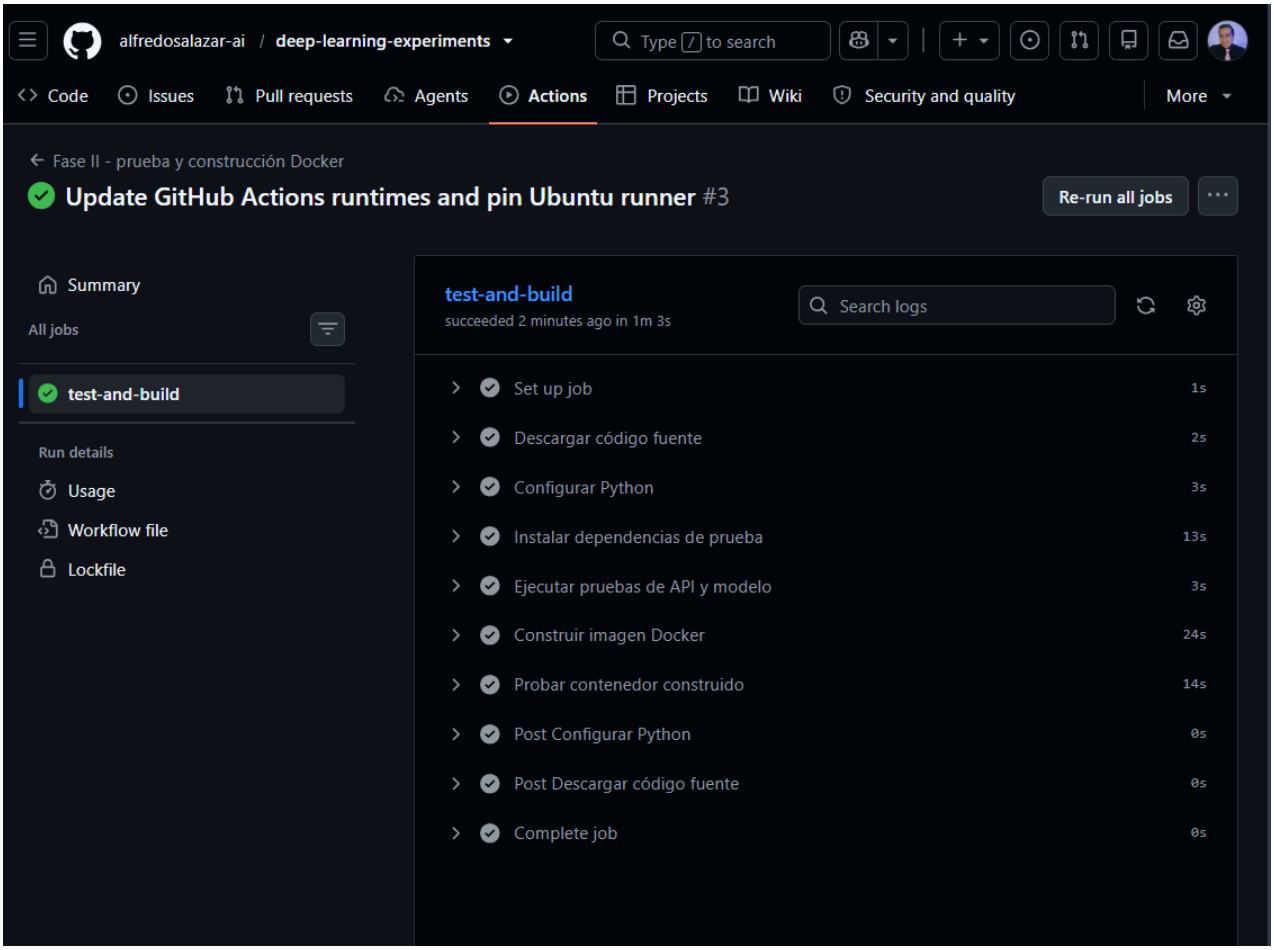


Figura 1. GitHub Actions: tercera ejecución exitosa del workflow de Fase II.

5. Despliegue en Render

El servicio se desplegó desde el repositorio GitHub, rama main, con runtime Docker y directorio del servicio FaseII. El registro de Render identifica el commit desplegado 4933ce629b4afa90e9160a757648af51ddf87227. El servicio quedó en estado “Deploy succeeded / Live” y publicó la URL https://chatbot-cima-fase2.onrender.com/.

Parámetro	Valor observado
Servicio	chatbot-cima-fase2
Plan	Free · USD 0/mes
Capacidad indicada	0.1 CPU · 512 MB RAM
Región	Oregon (US West)
Health Check Path	/health
Puerto de aplicación	10000 (Render asignó el puerto y Uvicorn escuchó en 0.0.0.0)
Auto-Deploy	On Commit
Variables de entorno	No se configuraron variables adicionales
Persistencia	No se configuró disco persistente

Los logs muestran que Render clonó el repositorio, construyó la imagen a partir de python:3.12-slim, instaló las dependencias fijadas y copió backend, frontend y modelos. Durante el arranque aparece “Modelo de CIMA cargado desde /app/modelos/Regresion_Logistica.joblib”, seguido de “Application startup complete” y Uvicorn escuchando en 0.0.0.0:10000. Render confirmó “Your service is live”.

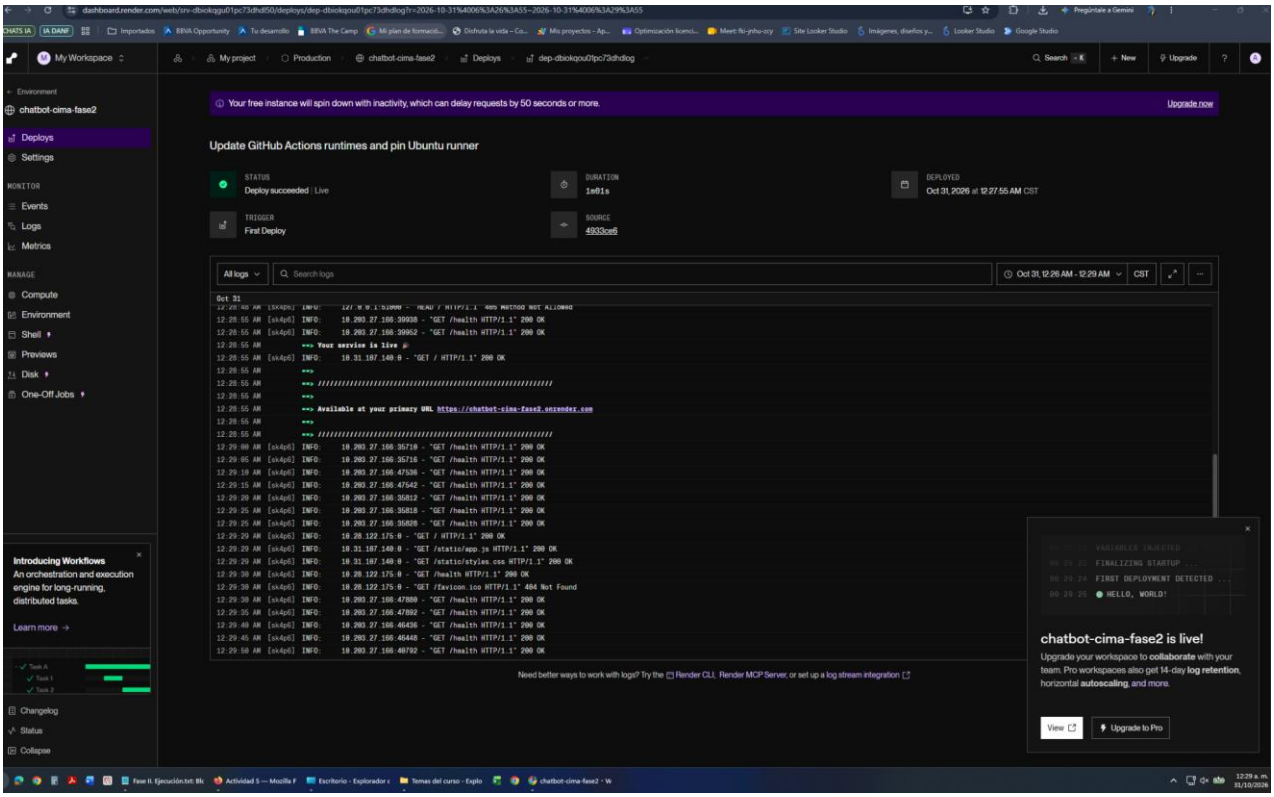


Figura 2. Logs de Render: construcción, carga del modelo, arranque y confirmación Live.

Los logs incluyen solicitudes periódicas GET /health con HTTP 200, realizadas por el mecanismo de comprobación de salud del servicio. También se observó una solicitud a /favicon.ico con HTTP 404; es un recurso accesorio no disponible y no afectó el funcionamiento de la aplicación. Una solicitud HEAD / inicial obtuvo 405 durante el arranque; el servicio continuó y fue declarado Live.

Limitación del plan gratuito Render Free puede suspender la instancia después de un periodo de inactividad y reactivarla con una nueva solicitud. El primer acceso tras la suspensión puede tardar. Este comportamiento debe considerarse al realizar la demostración.

6. Pruebas funcionales desde la interfaz web

Se ejecutaron cuatro consultas sintéticas desde la interfaz pública. Las clases y puntuaciones siguientes corresponden a los resultados visibles en las capturas.

ID	Consulta sintética (resumen)	Clase predicha	Puntaje IND.	Umbral	Decisión	Observación
WEB-01	Solicitud de confirmar un pago asociado al folio 12345.	INDIVIDUAL	0.2651	0.276334	Revisión humana	Clasificación INDIVIDUAL; puntaje inferior al umbral.

ID	Consulta sintética (resumen)	Clase predicha	Puntaje IND.	Umbral	Decisión	Observación
WEB-02	Consulta de un único pago CIE, referencia 987654321012 y folio 482731.	AMBIGUA	0.2388	0.276334	Revisión humana	La intención era individual; la clase predicha fue AMBIGUA.
WEB-03	Solicitud de revisión de paquete de 25 pagos CIE en archivo adjunto.	FUERA_ALCANCE	0.2356	0.276334	Revisión humana	La intención era PAQUETE; el modelo devolvió FUERA_ALCANCE.
WEB-04	Depósito pendiente de identificar, sin folio ni referencia.	AMBIGUA	0.2344	0.276334	Revisión humana	Resultado congruente con una consulta sin datos suficientes.

En las cuatro pruebas de interfaz, el puntaje de INDIVIDUAL quedó por debajo del umbral y la aplicación mostró revisión humana. Dos de las consultas no fueron clasificadas en la categoría sugerida por su intención (WEB-02 y WEB-03); ambas discrepancias se consideran hallazgos de calidad del clasificador.

7. Pruebas funcionales de API mediante Swagger UI

ID	Endpoint / escenario	HTTP	Resultado observado	Evaluación
API-01	GET /health	200	status=ok; model_loaded=true; service=cima-api.	Aprobada
API-02	POST /predict · consulta individual	200	INDIVIDUAL; puntaje 0.2651; umbral 0.276334; REVISION_HUMANA.	Aprobada
API-03	POST /predict · paquete de pagos	200	PAQUETE; puntaje 0.2357; umbral 0.276334; REVISION_HUMANA.	Aprobada
API-04	POST /predict · depósito sin folio	200	AMBIGUA; puntaje 0.2344216569; REVISION_HUMANA.	Aprobada
API-05	POST /predict · consulta turística	200	AMBIGUA (se esperaba FUERA_ALCANCE); puntaje 0.2445779859; REVISION_HUMANA.	Observación
API-06	POST /predict · texto vacío	422	string_too_short; longitud mínima 1 carácter.	Aprobada
API-07	POST /predict · campo texto ausente	422	missing; ubicación body.texto; Field required.	Aprobada
API-08	POST /predict · texto de 5,001 caracteres	422	string_too_long; máximo permitido 5,000 caracteres.	Aprobada

Las respuestas HTTP 422 de API-06, API-07 y API-08 son resultados esperados: acreditan que las validaciones de entrada se ejecutan antes de solicitar una predicción al modelo. API-05 devolvió una clasificación AMBIGUA en lugar de FUERA_ALCANCE; aunque la clasificación no coincidió con la intención, el sistema aplicó la revisión humana.

8. Evaluación de la política de enrutamiento

La versión desplegada comunica la puntuación de la clase INDIVIDUAL y el umbral 0.276334. En todas las pruebas funcionales de clasificación documentadas, el puntaje fue inferior al umbral y la decisión fue REVISION_HUMANA. No se obtuvo en esta batería un caso observado de aceptación automática por superar el umbral; por tanto, la ruta de aceptación automática no queda acreditada por estas evidencias.

Interpretación correcta Un puntaje inferior al umbral explica la derivación a revisión humana, pero no

debe interpretarse como probabilidad calibrada ni como certeza sobre la clase predicha. La clasificación del modelo y la decisión operativa son campos distintos.

9. Hallazgos, limitaciones y acciones recomendadas

Hallazgo	Implicación	Acción recomendada
WEB-02: consulta individual clasificada AMBIGUA.	El lenguaje/contexto de una solicitud individual puede no corresponder a los patrones aprendidos.	Ampliar ejemplos independientes y variados de INDIVIDUAL y AMBIGUA; evaluar por clase.
WEB-03: solicitud de paquete clasificada FUERA_ALCANCE.	Se observa confusión entre PAQUETE y FUERA_ALCANCE.	Incorporar casos de paquetes con diferentes redacciones, volúmenes y referencias.
API-05: consulta turística clasificada AMBIGUA.	El modelo no separa con robustez todo contenido claramente ajeno al dominio.	Ampliar ejemplos FUERA_ALCANCE y probar reglas deterministas de dominio antes/después del modelo.
Sin caso de aceptación automática observado.	La cobertura de aceptación de la política no fue demostrada en esta batería.	Diseñar un conjunto de prueba sintético representativo con puntajes sobre el umbral; registrar resultado real sin forzar el modelo.
Datos sintéticos y evidencia funcional limitada.	No permiten afirmar desempeño en producción ni generalización robusta.	Ampliar dataset independiente, especialmente AMBIGUA y FUERA_ALCANCE; evitar duplicados y fugas.

La retroalimentación metodológica previa se mantiene vigente: la comparación de modelos debe incluir pruebas estadísticas emparejadas cuando corresponda (por ejemplo, McNemar), los experimentos de búsqueda deben quedar trazados en MLflow y los datos deben ser suficientemente diversos. Esas actividades pertenecen a la evaluación y evolución del modelo; no se sustituyen por las pruebas de despliegue descritas aquí.

10. Conclusión

La Fase II acredita la integración y publicación de CHATBOT CIMA en un servicio web en la nube. GitHub Actions ejecutó correctamente las pruebas y la construcción Docker; Render construyó y ejecutó el contenedor, cargó el artefacto de Regresión Logística y mantuvo disponible el endpoint de salud. La interfaz y la API procesaron consultas sintéticas y los controles de validación rechazaron correctamente entradas inválidas.

La validación también identificó límites de clasificación en consultas individuales, de paquetes y fuera de alcance. La derivación a revisión humana mitigó la posibilidad de aceptación automática en los casos observados, pero no elimina la necesidad de mejorar el dataset y evaluar el modelo con una muestra más amplia e independiente. El prototipo es académico e informativo: no autoriza pagos, no confirma transacciones y no sustituye validaciones deterministas ni intervención humana.

Anexo A. Recursos y evidencias

Aplicación desplegada: https://chatbot-cima-fase2.onrender.com/

Swagger UI: https://chatbot-cima-fase2.onrender.com/docs

Health Check: https://chatbot-cima-fase2.onrender.com/health

Repositorio GitHub: https://github.com/alfredosalazar-ai/deep-learning-experiments